CSE563 — Unit V Numerical Worksheet

Algorithm Complexity Analysis

40 questions • CO5 • Solutions Booklet

Instructor Note

Full worked solutions for all 40 questions. Every complexity claim is justified, and for each recurrence the chosen method (iteration, recursion tree, substitution or master theorem) is named and the reason for choosing it is stated.

A. Counting Work and Asymptotic Notation (Q1–Q12)

Q1. Give the Θ complexity of the triple loop
`for i in range(n): for j in range(n): for k in range(n): x += 1`

Solution. Three independent nested loops, each running n times, so the innermost statement executes $n \times n \times n = n^3$ times.

$$\boldsymbol{\Theta(n^3)}.$$

Q2. Give the Θ complexity of: `for i in range(n): for j in range(i+1, n): x += 1`

Solution. For each i the inner loop runs $n - i - 1$ times, so the total is

$$\sum_{i=0}^{n-1}(n-i-1) = (n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}.$$

$$\boldsymbol{\Theta(n^2)}.$$

This counts each unordered pair once — it is $\binom{n}{2}$.

Q3. Give the Θ complexity of: `i = n; while i > 0: i = i // 4`

Solution. i takes $n, n/4, n/16, \ldots$ and the loop ends when $n/4^k < 1$, i.e. $k > \log_4 n$.

$$\boldsymbol{\Theta(\log n)}.$$

The base 4 is a constant factor ($\log_4 n = \log_2 n / 2$) and is discarded.

Q4. Give the Θ complexity of: `for i in range(n): j = 1; while j < n: j *= 2`

Solution. The outer loop runs n times. The inner loop doubles from 1 to n, running $\lceil \log_2 n \rceil$ times, and this count does not depend on i.

$$n \times \log_2 n = \boldsymbol{\Theta(n \log n)}.$$

Q5. Give the Θ complexity of: `for i in range(1, n+1): for j in range(1, n+1, i): x += 1`

Solution. For a fixed i, the inner loop steps by i and so runs about n/i times. The total is

$$\sum_{i=1}^{n} \frac{n}{i} = n \sum_{i=1}^{n} \frac{1}{i} = nH_n,$$

where H_n is the n-th harmonic number. Since $H_n = \Theta(\ln n)$ (bound the sum by $\int_1^n \frac{dx}{x} = \ln n$),

$$\boldsymbol{\Theta(\mathbf{n} \log \mathbf{n})}.$$

This is exactly the cost of the Sieve of Eratosthenes' inner structure.

Q6. Give the Θ complexity of two sequential blocks: one $\Theta(n^2)$ followed by one $\Theta(n \log n)$.

Solution. Sequential code adds: $T(n) = \Theta(n^2) + \Theta(n \log n)$. By the sum rule, $O(f) + O(g) = O(\max(f, g))$, and $n \log n = o(n^2)$, so the quadratic block dominates.

$$\boldsymbol{\Theta(\mathbf{n^2})}.$$

Q7. Prove from the definition that $7n^2 + 5n + 3 = O(n^2)$, giving explicit c and n_0.

Solution. For all $n \geq 1$ we have $n \leq n^2$ and $1 \leq n^2$, so

$$7n^2 + 5n + 3 \leq 7n^2 + 5n^2 + 3n^2 = 15n^2.$$

Taking $\mathbf{c = 15}$ and $\mathbf{n_0 = 1}$ satisfies $f(n) \leq cn^2$ for all $n \geq n_0$. Hence $f(n) = O(n^2)$. ■

Q8. Prove that $7n^2 + 5n + 3 = \Omega(n^2)$, and hence state the tight bound.

Solution. Every term is positive, so for all $n \geq 0$:

$$7n^2 + 5n + 3 \;\geq\; 7n^2.$$

Taking $c = 7$, $n_0 = 0$ gives $f(n) = \Omega(n^2)$. ■
Combined with the previous question, $7n^2 + 5n + 3 = \boldsymbol{\Theta(\mathbf{n^2})}$, with $c_1 = 7$, $c_2 = 15$, $n_0 = 1$.

Q9. Prove that $2^{n+1} = O(2^n)$ but $2^{2n} \neq O(2^n)$.

Solution. **First claim:** $2^{n+1} = 2 \cdot 2^n \leq 2 \cdot 2^n$, so $c = 2$, $n_0 = 0$ works. $2^{n+1} = O(2^n)$ — a constant *factor* in the exponent's offset is harmless. ■
Second claim: $2^{2n} = (2^n)^2 = 4^n$. Suppose $4^n \leq c\,2^n$ for all $n \geq n_0$. Dividing by $2^n > 0$ gives $2^n \leq c$ for all large n — impossible, since 2^n grows without bound. Choosing $n > \max(n_0, \log_2 c)$ gives a contradiction. $\mathbf{2^{2n} \neq O(2^n)}$. ■
Lesson: 2^{n+1} and 2^n differ by a constant factor; 2^{2n} and 2^n differ by an exponential factor.

Q10. Arrange in increasing order of growth: n^2, $n \log n$, 2^n, $\log n$, $n!$, n, $\sqrt{n}$, $n^{1.5}$, 1.

Solution.

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^{1.5} \prec n^2 \prec 2^n \prec n!$$

Justifications for the two least obvious steps:

- $n \log n \prec n^{1.5}$: divide by n to compare $\log n$ with $\sqrt{n}$; since $\lim \frac{\log n}{\sqrt{n}} = 0$ (L'Hôpital), the claim holds.
- $2^n \prec n!$: $n! = 1 \cdot 2 \cdots n$ has n factors, of which all but the first few exceed 2, so $n!/2^n \to \infty$.

Q11. Use the limit test to determine whether $f(n) = n \log n$ is O, Θ or Ω of $g(n) = n^2$.

Solution.

$$\lim_{n\to\infty} \frac{n \log n}{n^2} = \lim_{n\to\infty} \frac{\log n}{n} = 0$$

(by L'Hôpital: $\frac{1/n}{1} \to 0$).
A limit of 0 means f grows *strictly* slower, so

$$n \log n = o(n^2), \quad \text{hence } n \log n = O(n^2) \text{ but NOT } \Theta(n^2) \text{ or } \Omega(n^2).$$

Q12. An algorithm performs $T(n) = 4n^3 + 1000n^2 + 10^6$ operations. State the Θ class, and explain why the huge constant 10^6 is irrelevant asymptotically but may matter in practice.

Solution.

$$T(n) = \mathbf{\Theta(n^3)}.$$

$1000n^2 = o(n^3)$ and 10^6 is a constant, so both are dominated as $n \to \infty$.
Why the constant can still matter. Asymptotic notation describes behaviour for *large* n only. Here $4n^3$ overtakes 10^6 only when $n^3 > 250000$, i.e. $n > 63$; and $4n^3$ overtakes $1000n^2$ only when $n > 250$. So for inputs smaller than a few hundred the "lower-order" terms dominate the actual runtime. This is why, for example, real sorting libraries switch to insertion sort for small subarrays despite its worse asymptotic class.

B. Recurrence Relations (Q13–Q24)

Q13. Write the recurrence for binary search and for merge sort, and state the solution of each.

Solution. **Binary search:** one recursive call on half the array, $O(1)$ work to compare.

$$T(n) = T(n/2) + 1 \quad \Rightarrow \quad \mathbf{\Theta(\log n)}.$$

Merge sort: two recursive calls on halves, $\Theta(n)$ work to merge.

$$T(n) = 2T(n/2) + n \quad \Rightarrow \quad \mathbf{\Theta(n \log n)}.$$

Q14. Solve $T(n) = T(n-1) + c$ with $T(1) = c$, by iteration.

Solution. *Subtractive*, so iterate.

$$T(n) = T(n-1) + c = T(n-2) + 2c = \cdots = T(1) + (n-1)c = c + (n-1)c = nc.$$

$$\mathbf{\Theta(n)}.$$

Q15. Solve $T(n) = T(n-1) + n^2$ with $T(0) = 0$, by iteration.

Solution. *Subtractive*, so iterate:

$$T(n) = n^2 + (n-1)^2 + \cdots + 1^2 = \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}.$$

$$\boldsymbol{\Theta(n^3)}.$$

Q16. Solve $T(n) = 2T(n-1) + 1$ with $T(1) = 1$ (Tower of Hanoi), by iteration.

Solution.

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 4T(n-2) + 2 + 1 \\ &= 8T(n-3) + 4 + 2 + 1 \\ &\vdots \\ &= 2^k T(n-k) + (2^k - 1). \end{aligned}$$

At $k = n-1$: $T(n) = 2^{n-1}(1) + 2^{n-1} - 1 = 2^n - 1$.

$$\boldsymbol{\Theta(2^n)}.$$

Check: $T(3) = 7$, and three discs genuinely need 7 moves ✓.

Q17. Solve $T(n) = 2T(n-1) + 3$ with $T(0) = 1$, by iteration, and verify at $n = 3$.

Solution.

$$\begin{aligned} T(n) &= 2T(n-1) + 3 = 4T(n-2) + 6 + 3 = 8T(n-3) + 12 + 6 + 3 \\ &= 2^k T(n-k) + 3(2^{k-1} + \cdots + 2 + 1) = 2^k T(n-k) + 3(2^k - 1). \end{aligned}$$

At $k = n$: $T(n) = 2^n(1) + 3(2^n - 1) = 4 \cdot 2^n - 3$.

$$\boldsymbol{\Theta(2^n)}.$$

Verify $n = 3$: closed form $= 4(8) - 3 = 29$. Directly: $T(1) = 5$, $T(2) = 13$, $T(3) = 29$ ✓.

Q18. Use a recursion tree to solve $T(n) = 3T(n/3) + n$.

Solution. At level i there are 3^i nodes, each costing $n/3^i$, so every level costs $3^i \cdot \dfrac{n}{3^i} = n$. The subproblem size reaches 1 after $\log_3 n$ divisions, giving $\log_3 n + 1$ levels.

$$T(n) = n(\log_3 n + 1) = \boldsymbol{\Theta(n \log n)}.$$

(Confirmed by the master theorem: $d = \log_3 3 = 1$, $f = n = \Theta(n^1)$, Case 2.)

Q19. Use a recursion tree to solve $T(n) = 2T(n/2) + n^2$, and explain which level dominates.

Solution. Level i has 2^i nodes each costing $(n/2^i)^2 = n^2/4^i$, so the level total is

$$2^i \cdot \frac{n^2}{4^i} = \frac{n^2}{2^i}.$$

Summing over levels gives a geometric series:

$$T(n) = n^2 \left(1 + \tfrac{1}{2} + \tfrac{1}{4} + \dots\right) < 2n^2.$$

$$\boldsymbol{\Theta(n^2)}.$$

The root dominates: the work *halves* at each level, so the top level alone accounts for half the total. (Master theorem Case 3 gives the same answer.)

Q20. Prove by substitution (induction) that $T(n) = T(n/2) + 1$ is $O(\log n)$.

Solution. **Claim:** $T(n) \le c \log_2 n$ for all $n \ge 2$ and suitable c.
Basis ($n = 2$): $T(2) = T(1) + 1 = 2$ (taking $T(1) = 1$), and $c \log_2 2 = c \ge 2$ when $c \ge 2$ ✓.
Inductive step: assume the bound for all sizes below n. Then

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &\le c \log_2 \frac{n}{2} + 1 \\ &= c \log_2 n - c + 1 \\ &\le c \log_2 n \qquad \text{provided } c \ge 1. \end{aligned}$$

Taking $c = 2$ satisfies both requirements. Hence $T(n) = \mathbf{O(\log n)}$. ■

Q21. Solve the linear recurrence $a_n = 5a_{n-1} - 6a_{n-2}$ with $a_0 = 1$, $a_1 = 0$, and verify a_2.

Solution. Characteristic equation: $r^2 - 5r + 6 = 0 \Rightarrow (r-2)(r-3) = 0$, roots 2 and 3.

$$a_n = \alpha 2^n + \beta 3^n.$$

$n = 0$: $\alpha + \beta = 1$. $\quad n = 1$: $2\alpha + 3\beta = 0$. From the first, $\alpha = 1 - \beta$; substituting: $2 - 2\beta + 3\beta = 0 \Rightarrow \beta = -2$, so $\alpha = 3$.

$$\boxed{a_n = 3 \cdot 2^n - 2 \cdot 3^n}$$

Verify a_2: closed form $= 3(4) - 2(9) = 12 - 18 = -6$; recurrence $= 5(0) - 6(1) = -6$ ✓. Magnitude: $|a_n| = \Theta(3^n)$, governed by the larger root.

Q22. Solve $a_n = 4a_{n-1} - 4a_{n-2}$ with $a_0 = 1$, $a_1 = 6$ (note the repeated root).

Solution. Characteristic equation: $r^2 - 4r + 4 = 0 \Rightarrow (r-2)^2 = 0$, a **repeated root** $r = 2$. For a repeated root the general solution is

$$a_n = (\alpha + \beta n) 2^n.$$

$n = 0$: $\alpha = 1$. $\quad n = 1$: $(1 + \beta)2 = 6 \Rightarrow \beta = 2$.

$$\boxed{a_n = (1 + 2n) 2^n}$$

Verify a_2: closed form $= 5(4) = 20$; recurrence $= 4(6) - 4(1) = 20$ ✓.

Q23. State the Fibonacci recurrence, solve its characteristic equation, and explain why naive recursive Fibonacci is exponential.

Solution. $F_n = F_{n-1} + F_{n-2}$, $F_0 = 0$, $F_1 = 1$. Characteristic equation: $r^2 - r - 1 = 0$, roots

$$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618, \qquad \psi = \frac{1-\sqrt{5}}{2} \approx -0.618.$$

Binet's formula: $F_n = \frac{1}{\sqrt{5}}(\varphi^n - \psi^n)$. Since $|\psi| < 1$, the second term vanishes and $F_n = \Theta(\varphi^n)$.
Why naive recursion is exponential. The call tree for `fib(n)` recomputes `fib(n-2)` twice, `fib(n-3)` three times, and so on — the number of calls itself satisfies the Fibonacci recurrence, so it is $\Theta(\varphi^n) \approx \Theta(1.618^n)$. Memoisation reduces this to $\Theta(n)$ by storing each subproblem once (Unit VI).

Q24. For $T(n) = T(n-1) + \log n$, find the Θ class.

Solution. *Subtractive*, so iterate:

$$T(n) = \log n + \log(n-1) + \cdots + \log 1 = \log(n!).$$

By Stirling's approximation (or the sum–integral bound), $\log(n!) = \Theta(n \log n)$.

$$\boldsymbol{\Theta(\mathbf{n}\log\mathbf{n})}.$$

C. The Master Theorem (Q25–Q30)

Q25. Apply the master theorem to $T(n) = 9T(n/3) + n^2$, naming the case.

Solution. $a = 9$, $b = 3$, so $d = \log_3 9 = 2$ and $n^d = n^2$. $f(n) = n^2 = \Theta(n^2)$ — equal to n^d, so this is **Case 2**.

$$T(n) = \Theta(n^d \log n) = \boldsymbol{\Theta(\mathbf{n^2}\log\mathbf{n})}.$$

Q26. Apply the master theorem to $T(n) = 16T(n/4) + n$, naming the case.

Solution. $a = 16$, $b = 4$, $d = \log_4 16 = 2$, $n^d = n^2$. $f(n) = n = O(n^{2-1})$ with $\varepsilon = 1$ — polynomially smaller, so **Case 1** (the leaves dominate).

$$T(n) = \Theta(n^d) = \boldsymbol{\Theta(\mathbf{n^2})}.$$

Q27. Apply the master theorem to $T(n) = 3T(n/4) + n\log n$, including the regularity check.

Solution. $a = 3$, $b = 4$, $d = \log_4 3 \approx 0.7925$. $f(n) = n\log n$. Since $n\log n = \Omega(n^{0.7925+\varepsilon})$ for, say, $\varepsilon = 0.2$ (because $n^{0.99} = o(n\log n)$), **Case 3** is a candidate.
Regularity check: we need $af(n/b) \le kf(n)$ for some $k < 1$:

$$3 \cdot \frac{n}{4}\log\frac{n}{4} \;\le\; \frac{3}{4}n\log n,$$

which holds with $k = \frac{3}{4} < 1$ since $\log(n/4) < \log n$. ✓

$$T(n) = \Theta(f(n)) = \boldsymbol{\Theta(\mathbf{n}\log\mathbf{n})}.$$

Q28. Apply the master theorem to Strassen's recurrence $T(n) = 7T(n/2) + n^2$ and to Karatsuba's $T(n) = 3T(n/2) + n$, comparing each with the naive algorithm.

Solution. **Strassen.** $a = 7$, $b = 2$, $d = \log_2 7 \approx 2.807$. $f(n) = n^2 = O(n^{2.807-\varepsilon})$ with $\varepsilon \approx 0.8$ — **Case 1**.

$$T(n) = \boldsymbol{\Theta(n^{\log_2 7})} = \boldsymbol{\Theta(n^{2.807})}.$$

Naive matrix multiplication is $\Theta(n^3)$, so Strassen is asymptotically faster by replacing 8 half-size products with 7.
Karatsuba. $a = 3$, $b = 2$, $d = \log_2 3 \approx 1.585$. $f(n) = n = O(n^{1.585-\varepsilon})$ — **Case 1**.

$$T(n) = \boldsymbol{\Theta(n^{\log_2 3})} = \boldsymbol{\Theta(n^{1.585})}.$$

Schoolbook multiplication is $\Theta(n^2)$; Karatsuba replaces 4 half-size products with 3.
The shared idea: reducing the branching factor a lowers $\log_b a$, and since Case 1 gives $\Theta(n^{\log_b a})$, that directly lowers the exponent.

Q29. Explain why the master theorem cannot be applied to $T(n) = 2T(n/2) + n \log n$, and give the correct answer.

Solution. $a = 2$, $b = 2$, so $d = \log_2 2 = 1$ and $n^d = n$.
Compare $f(n) = n \log n$ with n. It is *larger*, so Case 2 does not apply. But Case 3 requires $f(n) = \Omega(n^{1+\varepsilon})$ for some $\varepsilon > 0$ — a *polynomial* gap. Here

$$\frac{f(n)}{n^d} = \log n,$$

which grows more slowly than n^ε for every $\varepsilon > 0$. The gap is only logarithmic, so the recurrence **falls in the gap between Cases 2 and 3** and the standard theorem is silent.
Correct answer (by the extended master theorem, $f = \Theta(n^d \log^k n)$ with $k = 1$ giving $\Theta(n^d \log^{k+1} n)$, or by a recursion tree in which each of the $\log n$ levels costs $\Theta(n \log n)$):

$$\mathbf{T(n) = \Theta(n \log^2 n)}.$$

Q30. State two further recurrences to which the master theorem does not apply, giving the reason in each case.

Solution. **(a)** $T(n) = T(n-1) + n$**.** The subproblem is $n - 1$, not n/b — the recurrence is **subtractive**, not divisive. The theorem's form $aT(n/b) + f(n)$ simply does not match. Solve by iteration: $\Theta(n^2)$.
(b) $T(n) = T(n/3) + T(2n/3) + n$**.** The subproblems have **unequal sizes** ($n/3$ and $2n/3$), whereas the theorem requires all a subproblems to be of the same size n/b. Solve by recursion tree: every level costs n and the tree has depth between $\log_3 n$ and $\log_{3/2} n$, giving $\Theta(n \log n)$.
(A third example: $T(n) = 2^n T(n/2) + n$, where $a = 2^n$ is not a constant.)

D. Complexity Classes and SAT (Q31–Q35)

Q31. Classify each as P or NP-complete, with a one-line justification: (a) sorting, (b) Hamiltonian cycle, (c) Euler circuit, (d) 3-colouring, (e) shortest path with non-negative weights.

Solution.

Problem	Class	Justification
(a) Sorting	**P**	merge sort runs in $\Theta(n \log n)$
(b) Hamiltonian cycle	**NP-complete**	a proposed cycle is verified in $O(n)$, but no polynomial algorithm is known
(c) Euler circuit	**P**	check every degree is even, $\Theta(n+m)$
(d) 3-colouring	**NP-complete**	a proposed colouring is checked in $O(m)$; deciding $\chi \leq 3$ is hard
(e) Shortest path ($w \geq 0$)	**P**	Dijkstra, $O((n+m)\log n)$

Note the instructive pairs: (b) versus (c), and (d) versus 2-colouring, which is in P.

Q32. Define P, NP, NP-hard and NP-complete, and state the relationship between them.

Solution.
P — decision problems solvable by a deterministic algorithm in polynomial time $O(n^k)$.
NP — decision problems whose *yes*-instances have a certificate verifiable in polynomial time.
NP-hard — at least as hard as every problem in NP — every NP problem reduces to it in polynomial time. It need not itself be in NP (the halting problem is NP-hard but undecidable).
NP-complete — in NP *and* NP-hard — the hardest problems *within* NP.
Relationships: P $\subseteq$ NP (solving quickly implies verifying quickly). NP-complete = NP$\cap$ NP-hard. Whether P = NP is open and is a Clay Millennium Prize problem; it is widely believed that P $\neq$ NP.

Q33. State precisely the two steps needed to prove a problem X is NP-complete, and explain why the reduction must go in a particular direction.

Solution. **Step 1 — show $X \in$ NP.** Exhibit a certificate for yes-instances and a polynomial-time verifier for it.
Step 2 — show X is NP-hard. Choose a *known* NP-complete problem Y and construct a polynomial-time reduction $Y \leq_p X$: a function mapping each instance of Y to an instance of X that preserves the yes/no answer.
Why the direction matters. $Y \leq_p X$ means "X is at least as hard as Y": if we could solve X quickly, we could solve Y quickly by translating and calling that solver. Since Y is NP-complete, everything in NP reduces to Y and hence to X.
Reducing the *other* way ($X \leq_p Y$) would only show that X is no harder than Y — which is true of every problem in NP and proves nothing about hardness. This reversal is the commonest error in such proofs.

Q34. Determine whether $\Phi = (a \vee b \vee c) \wedge (\neg a \vee \neg b) \wedge (\neg b \vee \neg c) \wedge (\neg a \vee \neg c)$ is satisfiable. If so, give a satisfying assignment.

Solution. Clauses 2–4 say that no *two* of a, b, c may both be true — i.e. at most one is true. Clause 1 says at least one is true. So exactly one of a, b, c must be true.
Try $(a, b, c) = (\mathrm{T}, \mathrm{F}, \mathrm{F})$:

Clause	Evaluation	Value
$(a \vee b \vee c)$	$\mathrm{T} \vee \mathrm{F} \vee \mathrm{F}$	T
$(\neg a \vee \neg b)$	$\mathrm{F} \vee \mathrm{T}$	T
$(\neg b \vee \neg c)$	$\mathrm{T} \vee \mathrm{T}$	T
$(\neg a \vee \neg c)$	$\mathrm{F} \vee \mathrm{T}$	T

Satisfiable, e.g. $(a, b, c) = (\mathrm{T}, \mathrm{F}, \mathrm{F})$. By symmetry $(\mathrm{F}, \mathrm{T}, \mathrm{F})$ and $(\mathrm{F}, \mathrm{F}, \mathrm{T})$ also work — exactly three satisfying assignments out of eight.

Q35. Show that $\Psi = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$ is unsatisfiable, using resolution.

Solution. **Resolution on y:** Resolve $(x \vee y)$ with $(x \vee \neg y) \Rightarrow (x)$. Resolve $(\neg x \vee y)$ with $(\neg x \vee \neg y) \Rightarrow (\neg x)$.
Resolution on x: resolve (x) with $(\neg x) \Rightarrow$ the **empty clause** $\square$.
Deriving the empty clause proves the formula is **unsatisfiable**. ■
Direct confirmation: the four clauses exclude (F, F), (T, F), (F, T) and (T, T) respectively — all four possible assignments.
Note: this formula is in 2-CNF, and 2-SAT is in P (solvable in $O(n + m)$ via the implication graph). General SAT and 3-SAT are NP-complete.

E. Searching and Sorting (Q36–Q40)

Q36. Trace binary search for the key 56 in $A = [3, 9, 14, 22, 31, 45, 56, 68, 79, 88, 95]$ (indices 0–10). State the number of comparisons and the worst case for $n = 11$.

Solution.

lo	hi	mid	A[mid]	Action
0	10	5	45	$45 < 56 \Rightarrow$ lo $= 6$
6	10	8	79	$79 > 56 \Rightarrow$ hi $= 7$
6	7	6	56	**found at index 6**

Three comparisons. Worst case for $n = 11$: $\lfloor \log_2 11 \rfloor + 1 = 3 + 1 = \mathbf{4}$ comparisons. Linear search would have needed 7 comparisons to reach index 6.

Q37. Trace the Lomuto partition of $A = [8, 3, 5, 10, 2, 7]$ with the last element as pivot. Give the array after each swap and the final pivot index.

Solution. Pivot $= 7$. i marks the end of the "$\leq$ pivot" region, initially -1; j scans indices 0–4.

j	$A[j]$	Test $A[j] \leq 7$	Array after
0	8	no	$[8, 3, 5, 10, 2, 7]$
1	3	yes: i=0, swap $A[0] \leftrightarrow A[1]$	$[3, 8, 5, 10, 2, 7]$
2	5	yes: i=1, swap $A[1] \leftrightarrow A[2]$	$[3, 5, 8, 10, 2, 7]$
3	10	no	$[3, 5, 8, 10, 2, 7]$
4	2	yes: i=2, swap $A[2] \leftrightarrow A[4]$	$[3, 5, 2, 10, 8, 7]$
end		swap pivot into $A[i{+}1] = A[3]$	$[3, 5, 2, \mathbf{7}, 8, 10]$

Final pivot index: 3. Left subarray $[3, 5, 2]$ (all ≤ 7), right $[8, 10]$ (all > 7). Quicksort now recurses on each.

Q38. Show the merge sort of $A = [38, 27, 43, 3, 9, 82, 10]$, giving the splitting and the merging steps.

Solution. **Split (top-down):**

$[38, 27, 43, 3, 9, 82, 10]$
$[38, 27, 43, 3] \mid [9, 82, 10]$
$[38, 27] \mid [43, 3] \qquad [9, 82] \mid [10]$
$[38]\,[27] \quad [43]\,[3] \qquad [9]\,[82] \quad [10]$

Merge (bottom-up):

$$[27, 38] \quad [3, 43] \qquad [9, 82] \quad [10]$$
$$[3, 27, 38, 43] \qquad [9, 10, 82]$$
$$[3, 9, 10, 27, 38, 43, 82]$$

Sorted result: $[3, 9, 10, 27, 38, 43, 82]$**.**
Merging two sorted lists of combined length k costs at most $k - 1$ comparisons. There are $\lceil \log_2 7 \rceil = 3$ levels of merging, each doing $O(n)$ work, giving $T(n) = 2T(n/2) + n = \Theta(n \log n)$ *regardless of the input order.*

Q39. State the best, average and worst case of quicksort, describe the input that triggers the worst case with a last-element pivot, and state how it is avoided.

Solution.

Case	Complexity	Recurrence
Best	$\Theta(n \log n)$	$T(n) = 2T(n/2) + n$ (balanced split)
Average	$\Theta(n \log n)$	expected over random pivots
Worst	$\Theta(n^2)$	$T(n) = T(n-1) + n$ (one side empty)

Worst-case input. With the *last element* as pivot, an **already-sorted** (or reverse-sorted) array is the worst case: every pivot is the maximum (or minimum) of its subarray, so each partition produces one empty side and one of size $n - 1$. Ironically the "easiest-looking" input is the worst.
Avoidance.

- **Randomised pivot** — choose uniformly at random, making the expected time $\Theta(n \log n)$ on *every* input, with no bad input class.
- **Median-of-three** — take the median of the first, middle and last elements, which defeats the sorted-input case in practice.
- **Introsort** (used by C++ `std::sort`) — switch to heap sort once the recursion depth exceeds $\sim 2 \log n$, guaranteeing $O(n \log n)$.

Q40. Prove that comparison-based sorting requires $\Omega(n \log n)$ comparisons in the worst case, and name one algorithm that sorts in linear time, stating the assumption that permits it.

Solution. **The decision-tree argument.** Model any comparison sort as a binary decision tree: each internal node is a single comparison with two outcomes, and each leaf corresponds to one possible output permutation. To sort correctly, the algorithm must be able to produce any of the $n!$ orderings, so the tree has at least $n!$ leaves.
A binary tree of height h has at most 2^h leaves, so $2^h \geq n!$, giving

$$h \;\geq\; \log_2(n!).$$

The height h is exactly the worst-case number of comparisons. By Stirling's approximation (or by bounding the sum),

$$\log_2(n!) = \sum_{k=1}^{n} \log_2 k \;\geq\; \sum_{k=n/2}^{n} \log_2 \frac{n}{2} \;=\; \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n). \qquad \blacksquare$$

Hence no comparison sort can beat $\Omega(n \log n)$; merge sort and heap sort attain it and are therefore asymptotically optimal.
Linear-time example: counting sort, $\Theta(n + k)$. **Assumption that permits it:** the keys must be **integers drawn from a bounded range** $[0, k]$ with $k = O(n)$. Counting sort never compares two keys — it uses each key as an *array index* — so the decision-tree model does not apply to it. (Radix sort extends this to d-digit keys in $\Theta(d(n + k))$.)